```
from sklearn.feature_extraction.text import CountVectorizer
corpus = [
    'This is the first document.',
    'This document is the second document.',
    'And this is the third one.',
    'Is this the first document?',
]
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(corpus)
vectorizer.get_feature_names_out()
print(X.toarray())
vectorizer2 = CountVectorizer(analyzer='word', ngram_range=(2, 2))
X2 = vectorizer2.fit_transform(corpus)
vectorizer2.get_feature_names_out()
```

```
[[0 1 1 1 0 0 1 0 1]
 [0 2 0 1 0 1 1 0 1]
 [1 0 0 1 1 0 1 1 1]
 [0 1 1 1 0 0 1 0 1]]
array(['and this', 'document is', 'first document', 'is the', 'is this',
       'second document', 'the first', 'the second', 'the third',
       'third one', 'this document', 'this is', 'this the'], dtype=object)
```

```
import requests
from bs4 import BeautifulSoup
from collections import Counter
import re

# 1. 모든 명언을 담을 리스트
all_quotes = []

# 2. 1페이지부터 10페이지까지 반복하여 크롤링
for i in range(1, 11):
    url = f"http://quotes.toscrape.com/page/{i}/"
    response = requests.get(url)
    soup = BeautifulSoup(response.text, 'html.parser')

    # 각 명언이 담긴 <span class="text"> 태그 추출
    quotes = soup.find_all('span', class_='text')

    for quote in quotes:
        # 양쪽 따옴표(u201c, u201d) 제거 후 리스트에 추가
        all_quotes.append(quote.text.strip('“ ” '))

# 3. 단어 빈도 분석을 위해 모든 명언을 하나의 텍스트로 합치기
all_text = " ".join(all_quotes).lower()

# 4. 특수문자 제거 및 단어 단위로 분리 (정규표현식 사용)
words = re.findall(r'₩w+', all_text)

# 5. 불용어(Stopwords) 제외 - (선택 사항: 'the', 'a', 'to' 등을 제외하고 싶을 경우
# 여기서는 순수하게 빈도수가 가장 높은 단어를 추출합니다.
word_counts = Counter(words)
```

```
# 6. 가장 출현 빈도가 높은 단어 5개 추출
top_5_words = word_counts.most_common(5)

# 결과 출력
print("--- 수집된 모든 명언 리스트 (일부) ---")
print(all_quotes[:5])
print("₩n--- 가장 출현 빈도가 높은 단어 5개 ---")
for word, count in top_5_words:
    print(f"{word}: {count}회")
```

```
--- 수집된 모든 명언 리스트 (일부) ---
['The world as we have created it is a process of our thinking. It cannot be changed

--- 가장 출현 빈도가 높은 단어 5개 ---
you: 103회
the: 75회
to: 75회
is: 69회
a: 69회
```

```
from sklearn.feature_extraction.text import CountVectorizer
import pandas as pd

# 1. CountVectorizer 설정 (단어가 있으면 1, 없으면 0으로 인코딩)
# binary=True 옵션이 지문에서 요구하는 0, 1 인코딩 방식입니다.
vectorizer = CountVectorizer(binary=True)

# 2. 전처리 및 변환 (all_quotes는 이전 단계에서 수집한 리스트)
X = vectorizer.fit_transform(all_quotes)

# 3. 데이터프레임 생성
# 컬럼명은 학습된 단어 사전(get_feature_names_out)을 사용합니다.
df_quotes_vector = pd.DataFrame(X.toarray(), columns=vectorizer.get_feature_names_(

# 결과 확인 (이미지처럼 100 rows × N columns 형태)
print(df_quotes_vector)
```

```
    000  10  abilities  about  above  absolutely  absurdities  accept  across  ₩
0     0   0          0      0      0           0            0       0       0
1     0   0          1      0      0           0            0       0       0
2     0   0          0      0      0           0            0       0       0
3     0   0          0      0      0           0            0       0       0
4     0   0          0      0      0           1            0       0       0
..  ...  ..        ...    ...    ...         ...          ...     ...     ...
95    0   0          0      0      0           0            0       0       0
96    0   0          0      0      0           0            0       0       0
97    0   0          0      0      0           0            0       0       0
98    0   0          0      0      0           0            0       0       0
99    0   0          0      0      0           0            0       0       0

    act  ...  writing  written  wrote  wrung  year  you  youer  your  yours  ₩
0     0  ...        0        0      0      0     0    0      0     0      0
1     0  ...        0        0      0      0     0    0      0     0      0
2     0  ...        0        0      0      0     0    0      0     1      0
3     0  ...        0        0      0      0     0    0      0     0      0
```

```
4     0  ...        0        0      0      0     0     0      0     0      0
..  ...  ...      ...      ...    ...    ...   ...   ...    ...   ...    ...
95    0  ...        0        0      0      0     0     1      0     0      0
96    0  ...        0        1      0      0     0     1      0     0      0
97    0  ...        0        0      0      0     0     0      0     0      0
98    0  ...        0        0      0      0     0     0      0     0      0
99    0  ...        0        0      0      0     0     0      0     0      0

    yourself
0          0
1          0
2          0
3          0
4          0
..       ...
95         0
96         0
97         0
98         0
99         0

[100 rows x 663 columns]
```

코딩을 시작하거나 AI로 코드를 생성하세요.

코딩을 시작하거나 AI로 코드를 생성하세요.